

Programación Orientada a Objetos

Unidad 1

Introducción a la Programación Orientada a Objetos (POO)

La Programación Orientada a Objetos resuelve cada programa desde el punto de vista de un Objeto. Dicho Objeto está formado por:

- **Datos** (llamados “**Atributos del Objetos**”)
- **Procedimientos u Operaciones** (llamados “**Métodos**”) que él mismo puede realizar con dichos datos.

Los Objetos pueden interactuar entre sí haciendo uso de sus métodos.

Paradigma de la Programación Orientada a Objetos

¿Qué es un Paradigma? Es un conjunto de prácticas y saberes que definen una disciplina científica durante un período específico.

El principio fundamental del paradigma de POO es construir un sistema en base a las entidades de un modelo elaborado a partir de un proceso de abstracción.

Para realizar esto, se debe diseñar todo el sistema utilizando el paradigma, comenzando por un análisis y un diseño orientado a objetos.

En general, las etapas a seguir son:

- Requerimientos
- Diseño
- Implementación
- Verificación y Depuración
- Mantenimiento

1. Desarrollo de Requerimientos:

Es la etapa que define QUÉ problema debe resolverse.

Su objetivo es identificar las necesidades, oportunidades o ideas que originan el software; siendo el resultado de esta operación un documento de especificación que establece la solución: sus restricciones y quienes participarán en su construcción.

En POO, se identifican los objetivos relevantes del problema y se crea un primer diagrama de clases.

2. Diseño:

A partir de los requerimientos, se busca determinar CÓMO será la solución técnica. Su objetivo es describir las clases (módulos) que integrarán el sistema y como se relacionarán entre sí; siendo el resultado de esto un documento que detalla el diseño que suele incluir diagramas de clases en UML y puede establecer los casos de prueba para la siguiente fase.

En POO, se define el comportamiento de los objetos, sus servicios y sus responsabilidades.

3. Implementación:

Es la etapa de construcción real del software.

Su objetivo es traducir el diseño a un lenguaje de programación.

Este paso es muy importante porque el programa debe mantener la estructura modular definida en el diseño; cada módulo de diseño debe corresponder a una unidad de código independiente.

4. Verificación y Depuración:

Esta etapa evalúa si lo que se programó realmente cumple con los requerimientos iniciales. Entonces:

- **Verificación:** Evalúa si el sistema satisface la especificación.
- **Depuración:** Es el proceso de corregir los errores detectados.

Se recomienda verificar cada unidad de código por separado y luego probar la integración de todas. Idealmente, parte de estas pruebas deben ser hechas por personas distintas a las que programaron.

5. Mantenimiento:

Es la etapa más larga, ya que ocurre una vez que el sistema está en uso.

Su objetivo es adaptar el software a medida que las necesidades del usuario cambian o crecen.

Un buen diseño modular es fundamental para que los cambios en una parte del sistema no afecten a los módulos que ya funcionan correctamente.

Programación Imperativa vs Programación Modular

1) Programación Imperativa: Se basa en dar órdenes directas a la computadora para que cambie su “estado” paso a paso. Se enfoca en el “cómo”.

Su estructura se basa en que el programa es una secuencia de instrucciones. Para organizarlo, se usa el refinamiento sucesivo (o sea, dividir un problema grande en pasos más pequeños).

El problema es que, en sistemas grandes, el código se vuelve difícil de seguir porque los datos y las funciones están mezclados. Si cambias algo en un paso, se podría romper todo el hilo.

2) Programación Modular: Es una evolución que busca resolver el caos de la programación imperativa pura. En lugar de un solo bloque de código, dividimos el sistema en piezas independientes llamadas “módulos”. Ahora, su enfoque es la “organización”.

Brinda independencia, porque cada módulo tiene una tarea específica y se comunica con los demás a través de interfaces claras. Además, un módulo no necesita saber como funciona el otro por dentro, solo qué resultado le da.

La diferencia clave es que el desarrollo de software de calidad requiere extensibilidad y reusabilidad:

- **En la imperativa,** es muy difícil sacar un pedazo de código y usarlo en otro programa porque está muy ligado al resto.
- **En la modular,** podríamos llevarnos un módulo entero a otro proyecto.

En la POO, los “módulos” son las clases. Cada clase es un módulo que no solo agrupa funciones, sino que agrupa los datos (atributos) con sus acciones (métodos).

UML (Lenguaje de Modelado Unificado)

UML es un lenguaje que permite la visualización, especificación y documentación de sistemas orientados a objetos. No es un lenguaje de programación, sino un **lenguaje de modelado** que ayuda a que todo el equipo de desarrollo entienda la estructura del sistema sin distraerse con los detalles del código.

UML define 13 diagramas para describir distintas perspectivas del sistema.

- **Diagramas de Secuencia.**
- **Diagramas de Comunicación.**
- **Diagramas de Clases.**
- **Diagramas de Objetos.**
- Diagramas de Tiempo.
- Diagramas de Descripción de la Interacción.
- Diagramas de Paquetes.
- Diagramas de Escrituras Compuestas.
- Diagramas de Componentes.
- Diagramas de Despliegue.
- Diagramas de Actividades.
- Diagramas de Estados.

Elementos Básicos de la Orientación a Objetos:

1) Objetos

Un objeto del problema es una unidad atómica que encapsula estado y comportamiento. La encapsulación permite una alta cohesión y bajo acoplamiento.

Los objetos son entidades (física o conceptual) que poseen atributos y comportamientos, los cuales están determinados por el conjunto de servicios y responsabilidades que el objeto debe brindar y asumir respectivamente.

- **Atributos de un Objeto:** Describen la abstracción de las características individuales que posee el objeto.
- **Comportamientos:** Representan las operaciones que pueden ser realizadas por el objeto.

Existen 2 objetos: El Objeto del Problema y el Objeto del Software.

1. **Objeto del Problema:** Es el ente real. Si el problema es un hospital, el paciente que está sentado en la sala de espera es un objeto del problema.
2. **Objeto de Software:** Es la representación de ese paciente dentro de la computadora. Existe en la memoria RAM mientras el programa corre.

El **estado** de un objeto está determinado por el conjunto de valores que tienen sus atributos en un momento dado. No es estático, cambia a lo largo del tiempo. El estado está encapsulado, lo que significa que otros objetos no pueden modificar sus atributos directamente, sino pedirle al objeto que lo haga a través de sus servicios.

El **comportamiento** es el conjunto de acciones y relaciones de ese objeto. Define como reacciona el objeto ante los mensajes que recibe de otros. Cada objeto tiene una responsabilidad específica y a veces, el comportamiento suele depender del estado.

La **identidad** es la propiedad que permite distinguir a un objeto de todos los demás, incluso si tienen el mismo estado (mismos atributos). Mientras el estado puede cambiar, la identidad no cambia nunca durante la vida del objeto.

Actores

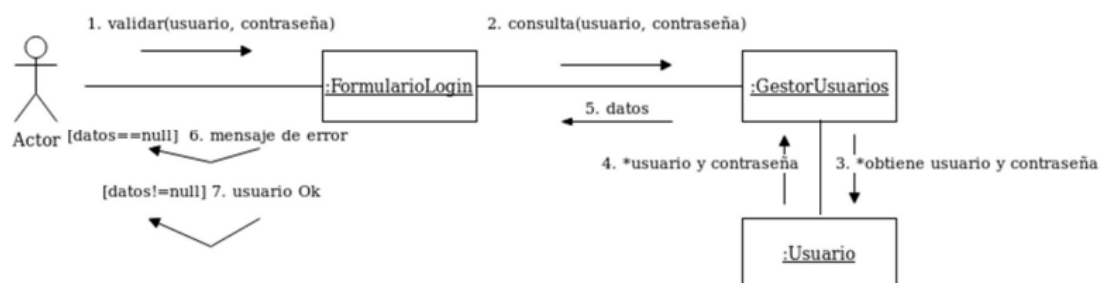
Un actor es cualquier entidad externa que interactúa con el sistema que estamos diseñando. El actor está fuera del software, es quién “aprieta los botones” o “recibe la información”.

No siempre es una persona, puede ser un humano (Médico, Bibliotecario) o también otro sistema o un dispositivo de Hardware.

Diagramas de Comunicación

Muestran la organización de los objetos que participan en una interacción y los mensajes que se intercambian, numerados en un orden secuencial.

A diferencia del diagrama de secuencia, el de comunicación se centra en las relaciones de los objetos. Éstos están conectados por líneas (asociaciones) y flechas pequeñas al lado de esas líneas que indican el mensaje.



- El * precediendo un mensaje, representa iteración.
- Los corchetes [] con texto entre ellos, expresan una condición de un condicional, como if... then...

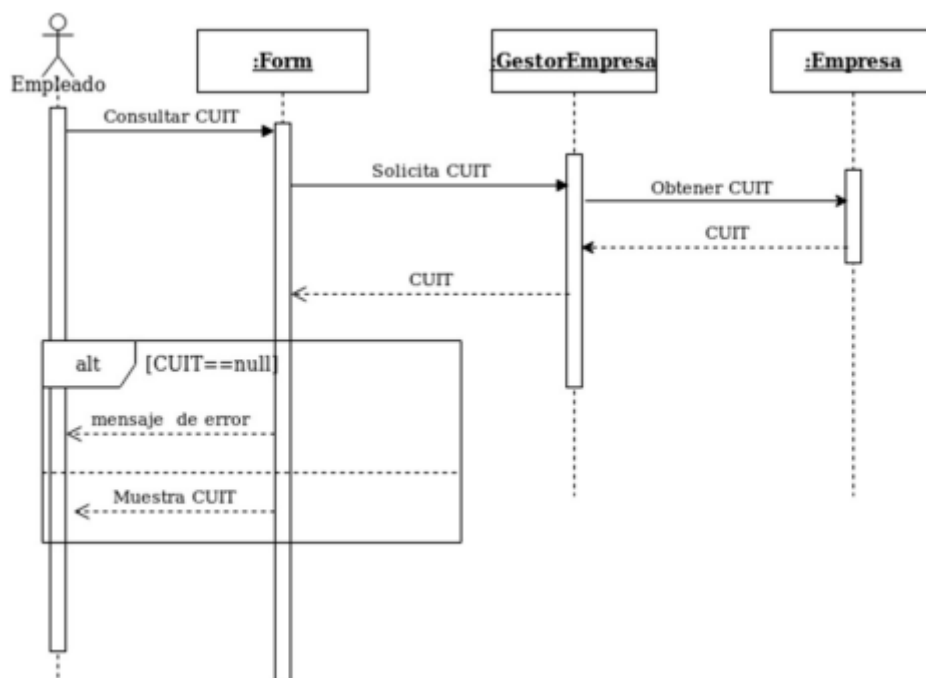
Diagramas de Secuencia

Muestra las interacciones entre objetos enfocándose en el orden cronológico en que se intercambian los mensajes.

Cada objeto se visualiza mediante un rectángulo con una línea de vida vertical, sobre la cual aparecen barras de activación que indican cuando el objeto está ejecutando una tarea.

La secuencia temporal se lee de arriba hacia abajo, permitiendo observar claramente qué objeto inicia una acción, quién responde y cuánto dura cada participación dentro de un escenario específico del sistema.

Ej: Un empleado solicita a través de un formulario el CUIT de una Empresa. El formulario muestra al empleado el CUIT o un mensaje de error si lo encontró.



Existen los “Fragmentos Combinados”, que son mecanismos que se dibujan como marcos sobre el diagrama de secuencia para agrupar mensajes y definir una lógica condicional o iterativa.

Cada uno tiene un operador en la esquina superior izquierda que indica el tipo de lógica que se está aplicando:

- **Fragmento Alternativa (ALT):** Funciona como un if-else para mostrar caminos que se excluyen entre sí según la condición.
- **Fragmento de Iteración (LOOP):** Funciona para indicar que una serie de mensajes se repiten mientras se cumpla una guarda.

Objetos

Existen objetos que no pertenecen al dominio del problema, pero surgen para presentar una interfaz al actor o para controlar la lógica interna de la misma.

1) Objeto de Interfaz (Boundary): Es el mediador entre el sistema y el mundo exterior, ya sea un usuario humano o un sistema externo. Su función es captar las peticiones y mostrar los resultados, pero no decide como procesar los datos ni sabe cómo guardarlos.

Notación



Relaciones Permitidas:



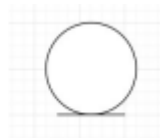
2) Objeto de Control (Controllers): Actúa como el coordinador, sirviendo de puente entre la interfaz y las entidades. Su responsabilidad es organizar y controlar la lógica requerida para alcanzar un objetivo.

Notación

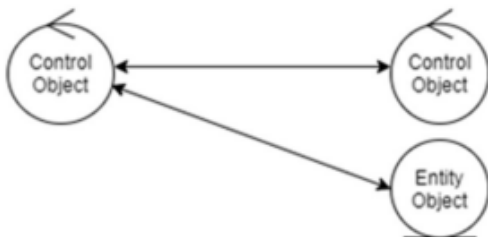


3) Objeto de Entidad (Entity): Guardan los datos y la lógica más básica, pero se mantienen como piezas reutilizables en diferentes partes del sistema. Modelan información asociada a un fenómeno o concepto, como persona u objeto.

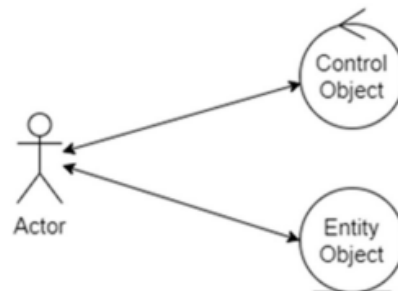
Notación



Relaciones Permitidas:



Relaciones Prohibidas:



2) Clases

Es un molde, modelo o plantilla que describe las características y comportamientos que compartirán todos los objetos de un mismo tipo.

Es una definición abstracta que no ocupa espacio real en la memoria del sistema hasta que se “instancia”, o sea, hasta que creamos un objeto basado en ese plano.

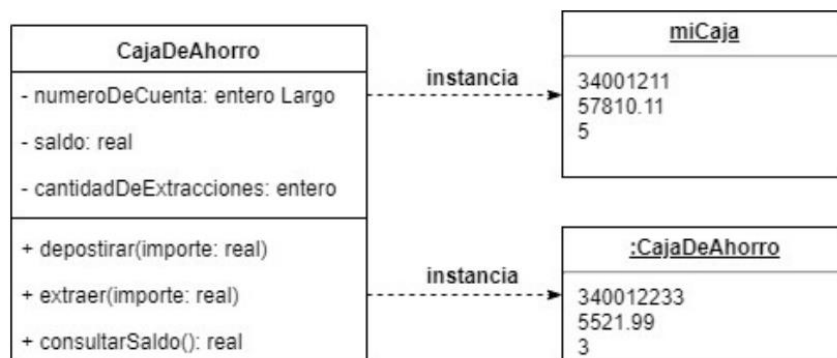
Se compone de 2 elementos clave:

- **Atributos (el estado):** Son las variables que guardan la información del objeto.
- **Métodos o Servicios (el comportamiento):** Son las funciones que definen qué acciones puede realizar el objeto.

Una buena clase debe cumplir con el *principio de encapsulamiento*, ocultando sus datos internos y permitiendo el acceso solo a través de una interfaz clara de mensajes, lo que garantiza la integridad y seguridad del software.

Cada clase se representa en un rectángulo con 3 compartimientos:

- Nombre de la Clase
- Atributos de la Clase: Que deben ser privados.
- Métodos de la Clase: Que deben ser públicos.



Además, mediante la *herencia*, las clases pueden organizarse en jerarquías, permitiendo que una clase “hija” (subclase) herede las propiedades de una clase “padre” (superclase). Esto facilita la reutilización de código, ya que, podemos definir comportamientos generales en una clase base y luego especializarnos en clases más específicas sin tener que reescribir todo desde cero.

3) Mensajes y Métodos

Un mensaje es el mecanismo de comunicación que utilizan los objetos para interactuar entre sí: un objeto emisor envía un mensaje a un objeto receptor solicitándole que ejecute uno de sus métodos.

Por otro lado, un método es la función o acción definida dentro de una clase que determina lo que ese objeto “sabe hacer”; es el bloque de código que se ejecuta para realizar una tarea específica como “activar()” o “recargar()”. Los métodos tienen la capacidad de acceder y modificar el estado del propio objeto.

Los métodos son los que se ejecutan para responder a un mensaje;
El mensaje es la llamada o invocación a un método.

Variables de Instancia y Variables de Clases

Las variables de instancia son atributos definidos dentro de una clase que pertenecen de forma única a cada objeto o “instancia” individual.

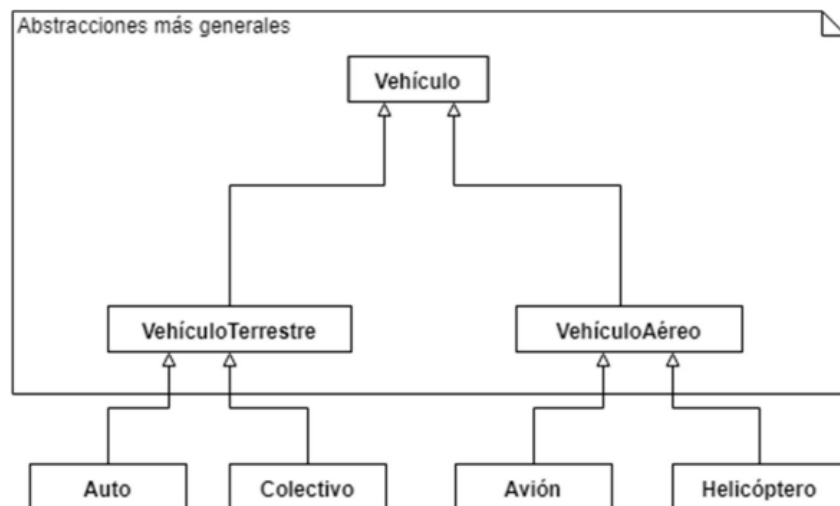
Por otro lado, las variables de clase son atributos que pertenecen a la clase en sí y no a un objeto específico. Existe una única copia de esta variable para todos los objetos de esa clase, lo que significa que es compartidas. Si un objeto modifica una variable de clase, el cambio se refleja inmediatamente para todos los demás objetos.

Mientras que las variables de instancia definen la identidad y estado de cada objeto, las variables de clases permiten la comunicación y ahorro de recursos.

4) Herencia

Permite crear nuevas clases a partir de otras ya existentes. Permite a una subclase adquirir automáticamente todos los atributos y métodos de una superclase, fomentando la reutilización de código y evitar la redundancia.

Generalización: Es el proceso de diseño “de abajo hacia arriba”, donde identificamos características comunes de varias clases y las agrupamos en una clase superior más abstracta.



Especialización: Es el proceso “de arriba hacia abajo”, donde tomamos una clase general y creamos variantes más específicas para cubrir necesidades particulares. Mediante la especialización, podemos agregar funciones nuevas creando subclases, sin tocar ni romper el código que ya está funcionando en la clase superior.



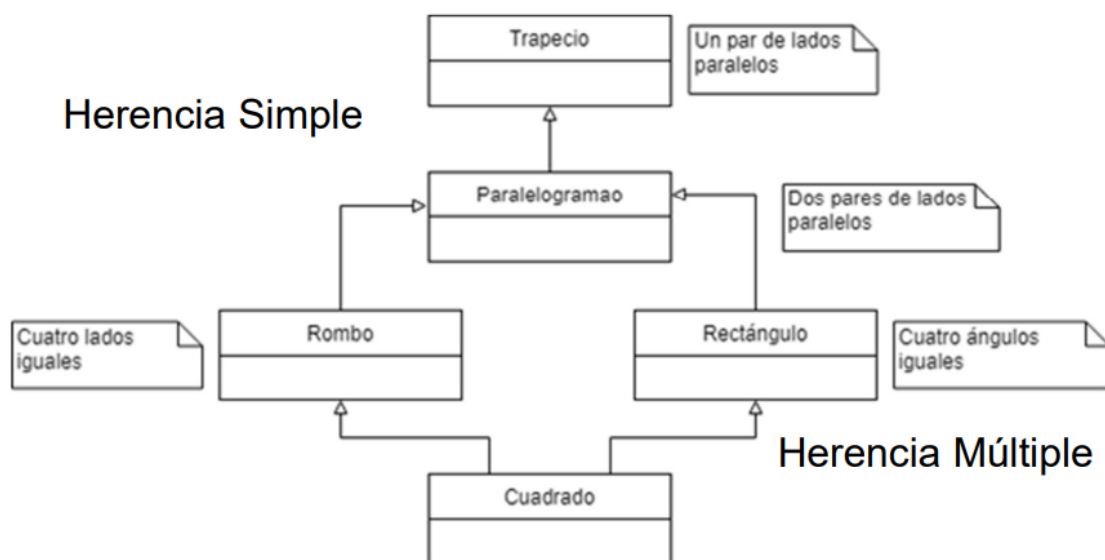
Clase Abstracta vs Clase Concreta

Clase Abstracta es una que no puede ser instanciada, o sea, no se pueden crear objetos directamente de ella. Se utiliza como un molde incompleto que define atributos o métodos comunes para sus subclases, pero deja ciertos comportamientos vacíos para que cada subclase los implemente a su manera.

Clase Concreta es aquella que está completamente definida y de la cual si se pueden crear objetos en la memoria. Estas clases heredan la estructura de la clase abstracta y completan los detalles que faltaban, volviéndolas funcionales para el sistema.

- La **diferencia fundamental** radica en la **utilidad**: Mientras que la clase abstracta sirve para organizar y jerarquizar el código evitando la repetición; las clases concretas son las que “realmente trabajan” durante la ejecución del programa.

Tipos de Herencia



La **Herencia Simple** ocurre cuando una subclase tiene una única superclase directa de la cual hereda sus atributos y métodos, mientras que la **Herencia Múltiple** permite que una subclase herede características de 2 o más superclases al mismo tiempo.

Relaciones entre Clases

Según el tipo de acoplamiento que las clases presenten se pueden distinguir distintos tipos de relaciones:

1. Asociación.
2. Agregación.
3. Generalización / Especialización / Herencia.

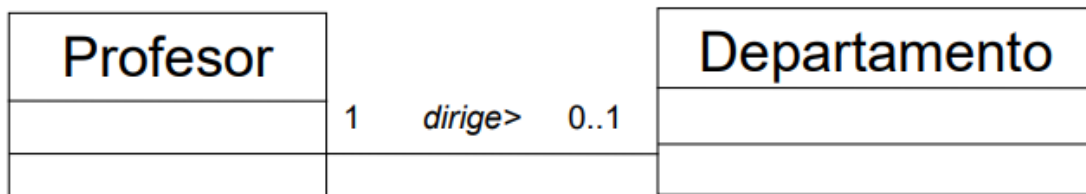
1. Asociación

Es una relación de colaboración, donde se describe la conexión entre 2 o más clases, indicando que los objetos de una interactúan con los objetos de la otra de forma sostenida.

En los diagramas de clases, la asociación se representa con una línea sólida que une a ambas entidades y puede incluir elementos críticos como la **multiplicidad y el rol**.

La **Multiplicidad** indica cuántos objetos de un lado se relacionan con cuántos objetos del otro. Sus valores más comunes son:

- **1**: Uno y solo uno.
- **0..1**: Cero o uno solo.
- **m..n**: De m a n (m y n son enteros naturales)
- *** o 0..***: De cero a varios.
- **1..***: De uno a varios.



*Un departamento tiene un solo profesor que lo dirige
Un profesor puede dirigir uno o ningún departamento.*

Clases de la Asociación

1. Clase Asociación: Se utiliza cuando una relación entre 2 clases tiene propiedades o comportamientos propios que no pertenecen lógicamente a ninguna de las dos partes por separado. En lugar de una simple línea, se representa como una clase conectada a la línea de asociación mediante una línea de puntos.

Ejemplo: Teniendo 2 clases: *Médico* y *Paciente*, genera una *Consulta*. Los datos de esa consulta (fecha, hora, diagnóstico), no son atributos del médico ni del paciente, sino que existen únicamente porque ambos se relacionan con un evento específico.

2. Clase que Modela una Asociación: Se aplica cuando el lenguaje de programación no soporta el concepto de “Clase Asociación”, por lo que, el diseñador transforma la relación en una clase intermedia concreta que tiene asociaciones simples hacia las otras dos.

La diferencia entre ambas radica en que la Clase Asociación enfatiza la relación entre objetos, la Clase que Modela una Asociación está pensada para la implementación y la gestión de datos en la base de datos o la memoria.

2. Agregación

Reutilización y Extensión

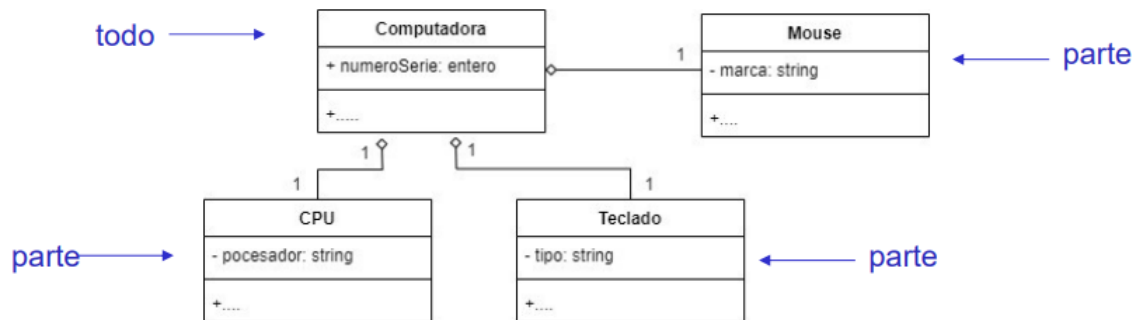
Reutilización: Es el principio que permite que el código ya escrito y probado se usa en nuevos contextos sin tener que re-escribirlo. Mientras que la herencia ofrece reutilización por especialización, la agregación y composición ofrecen reutilización por delegación: un objeto le pide a sus partes que hagan el trabajo.

Extensión: Es la capacidad de un sistema para incorporar nuevas funcionalidades con un impacto mínimo en el código existente. Principalmente se logra a través de la Herencia.

Agregación (propriadamente dicha)

Es un tipo especial de asociación que representa una relación donde una clase (el todo) contiene a otras (las partes), pero estas pueden existir de forma independiente.

Gráficamente, se representa colocando un rombo del lado de la clase agregado.

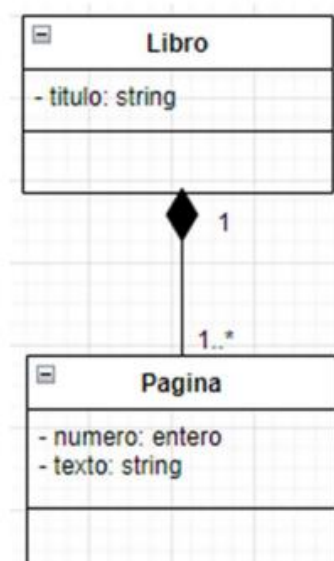


Computadora actúa como el "todo" (contenedor) y el **Mouse**, **Teclado** y la **CPU** son las "partes" (contenidos). El acoplamiento es débil, porque si la computadora se rompe o se elimina del sistema, el mouse o el teclado siguen existiendo como objetos válidos que podrían conectarse a otra máquina.

Los tiempos de vida no están ligados; se puede crear un **Mouse** hoy y asociarlo a una **Computadora** mañana, o destruir la **Computadora** sin que el objeto **Mouse** desaparezca de la memoria del programa. Por eso la forma del rombo es vacía (sin relleno).

Composición

Es una forma mucho más fuerte de agregación donde la existencia de las partes depende estrictamente de la existencia del todo. Si el objeto principal se destruye, sus partes también desaparecen.



Libro es el "todo" y las Paginas son las "partes". Si el libro se destruye, las páginas que lo componen desaparecen con él, ya que sus tiempos de vida están estrechamente acoplados.

Por esto es que el rombo no aparece vacío, sino totalmente relleno, garantizando que el objeto principal tenga el control total sobre la creación y destrucción de sus componentes.

Conceptos Clave

1. Encapsulamiento: Es el principio que consiste en ocultar el estado interno de un objeto y protegerlo del acceso directo desde el exterior.

Se basa en agrupar los atributos y métodos en una misma unidad, permitiendo que el objeto sea una “caja sólida” que solo expone una interfaz segura. Esto se logra definiendo los atributos como privados y los métodos como públicos, obligando a los demás objetos a usar mensajes para interactuar.

2. Abstracción: Es el proceso mental que permite identificar las características y comportamientos esenciales de un objeto, ignorando los detalles irrelevantes para el contexto actual.

3. Polimorfismo: Es la capacidad que tienen diferentes objetos de responder de distintas maneras al mismo mensaje. Está ligado a la herencia: Si la superclase define un método, las subclases pueden ejecutar ese mismo mensaje, pero cada uno lo hará con una implementación propia.

4. Persistencia: Es la propiedad que permite que el estado de un objeto trascienda en el tiempo, sobreviviendo incluso después de que el programa se cierre o la computadora se apague.

Esto implica guardar los datos generalmente en una base de datos o un archivo.

Gracias a esto, el “mundo de los objetos” puede ser reconstruido exactamente como estaba en la sesión anterior, asegurando que la información crítica no se pierda al liberar la memoria RAM.